

Aprendizado de Máquina B

Exercícios — Segmentação Semântica

Aluno: Lucas Affonso Pires | Matrícula: 2023028420

Questão 1: O que é segmentação semântica?

Resposta:

Segmentação semântica é a tarefa de classificar cada pixel de uma imagem em uma categoria predefinida. O resultado é um mapa de segmentação com as mesmas dimensões espaciais da imagem original, no qual cada pixel recebe um rótulo de classe (ex: pessoa, carro, estrada, céu). Diferentemente da detecção de objetos, a segmentação semântica não distingue instâncias individuais do mesmo objeto — todos os pixels de uma determinada classe recebem o mesmo rótulo.

Questão 2: Por que segmentação semântica é uma tarefa de predição densa e detecção de objetos é esparsa?

Resposta:

Segmentação semântica é considerada uma tarefa de predição densa porque a rede produz uma saída para cada pixel da imagem: em uma imagem de $N \times M$ pixels, são geradas $N \times M$ previsões de classe. Toda a grade espacial da imagem é preenchida com previsões.

Detecção de objetos, por outro lado, é uma predição esparsa: a rede prevê apenas um conjunto reduzido de bounding boxes e classes para regiões de interesse, sem classificar cada pixel individualmente. A maioria dos pixels da imagem não recebe uma predição direta — apenas as regiões onde objetos são detectados.

Questão 3: Diferenças entre classificação de imagens, detecção de objetos, segmentação semântica e segmentação de instâncias.

Resposta:

- Classificação de imagens: atribui um único rótulo de classe à imagem inteira. Não localiza nem segmenta objetos. Saída: vetor de probabilidades por classe.
 - Detecção de objetos: localiza instâncias de objetos com bounding boxes retangulares e classifica cada uma. Saída: lista de caixas (x, y, w, h) com classe e confiança associadas.
 - Segmentação semântica: classifica cada pixel da imagem em uma das categorias predefinidas. Não distingue instâncias individuais do mesmo objeto (ex: dois carros são pintados com a mesma cor). Saída: mapa de rótulos pixel a pixel.
 - Segmentação de instâncias: combina detecção de objetos e segmentação — classifica cada pixel e distingue instâncias individuais do mesmo objeto (cada carro recebe uma máscara única). Saída: máscara binária individual por instância detectada.
-

Questão 4: Quais são algumas aplicações da segmentação semântica?

Resposta:

- Veículos autônomos: identificação pixel a pixel de pista, pedestres, sinais de trânsito, obstáculos e outros carros para tomada de decisão em tempo real

- Diagnóstico médico por imagem: segmentação de órgãos, tecidos ou tumores em exames de ressonância magnética, tomografia e ultrassom, auxiliando o diagnóstico médico
 - Análise de imagens de satélite e aéreas: mapeamento de uso e cobertura do solo, identificação de florestas, corpos d'água, construções urbanas e áreas agrícolas
 - Robótica e visão computacional industrial: identificação de superfícies e objetos para navegação autônoma e controle de qualidade em linha de produção
 - Realidade aumentada: separação de plano de fundo e objetos para inserção de elementos virtuais de forma contextualizada
-

Questão 5: Como criar um mapa de probabilidades para segmentação semântica com 2 classes.

Resposta:

A rede processa a imagem de entrada e , por meio de camadas convolucionais e operações de upsampling, gera um tensor de saída com dimensão $H \times W \times C$, onde H e W são as dimensões espaciais da imagem e C é o número de classes (no caso, $C = 2$).

Para cada pixel (i, j) , o tensor de saída contém um vetor de C valores (logits). Aplica-se a função softmax sobre esses valores:

$$P(\text{classe}_k \mid \text{pixel}(i, j)) = \exp(z_k) / \sum_c \exp(z_c)$$

O resultado é um mapa de probabilidades $H \times W \times C$, onde cada posição (i, j, k) representa a probabilidade estimada de o pixel (i, j) pertencer à classe k . O pixel é então atribuído à classe com maior probabilidade (argmax). Para 2 classes binárias, pode-se usar sigmoid em vez de softmax, obtendo diretamente a probabilidade de pertencer à classe positiva.

Questão 6: Como transformar uma arquitetura de classificação em rede de segmentação semântica.

Resposta:

O processo envolve duas modificações principais:

- Substituição das camadas Fully Connected por camadas convolucionais 1×1 : as camadas FC no topo da rede de classificação (ex: VGG, ResNet) são substituídas por convoluções, tornando a rede completamente convolucional (Fully Convolutional Network — FCN). Isso permite que a rede aceite imagens de qualquer tamanho e produza mapas de ativação espaciais.
- Adição de camadas de upsampling (deconvolução / transposed convolution): como as operações de pooling e convolução com stride reduzem a resolução espacial, é necessário restaurá-la por meio de operações de upsampling para gerar um mapa de predições do mesmo tamanho da imagem original.

Possível desvantagem: o downsampling agressivo das camadas iniciais faz com que detalhes finos de bordas e regiões pequenas sejam perdidos. O upsampling posterior pode não recuperar essas informações com precisão suficiente, resultando em segmentações com bordas grosseiras ou imprecisas. Isso é mitigado por técnicas como skip connections (U-Net) ou dilated convolutions (DeepLab).

Questão 7: Por que uma rede Fully Connected não funciona em segmentação semântica/de instância?

Resposta:

- Perda de informação espacial: camadas FC tratam a entrada como um vetor unidimensional, destruindo completamente a estrutura espacial (posição relativa dos pixels). Segmentação requer saber onde cada pixel está.
 - Invariância à posição: operações FC são invariantes à posição, tornando impossível associar previsões a localizações específicas na imagem.
 - Tamanho de entrada fixo: camadas FC requerem que todas as imagens tenham exatamente o mesmo tamanho, o que é impraticável em aplicações reais.
 - Saída não espacial: a saída de uma FC é um vetor de probabilidades globais, não um mapa espacial. Não há como mapear diretamente uma previsão a um pixel específico da imagem de saída.
-

Questão 8: Quais são algumas arquiteturas comuns usadas em segmentação semântica?

Resposta:

- FCN (Fully Convolutional Network): primeira arquitetura clássica baseada em conversão de classificadores em redes totalmente convolucionais com upsampling
 - U-Net: arquitetura encoder-decoder com skip connections simétricas que conectam camadas do encoder ao decoder, preservando informações espaciais de alta resolução. Muito usada em imagens médicas.
 - SegNet: similar ao U-Net, usa max-pooling indices para guiar o upsampling, eficiente em memória
 - DeepLab (v1 a v4): utiliza dilated (atrous) convolutions para ampliar o campo receptivo sem reduzir resolução, e Atrous Spatial Pyramid Pooling (ASPP) para capturar contexto multi-escala
 - PSPNet (Pyramid Scene Parsing Network): usa um módulo de pooling piramidal para agregar contexto global e local de diferentes escalas
-

Questão 9: Objetivo do downsampling e upsampling em uma Fully Convolutional Network.

Resposta:

Downsampling (encoder): reduz progressivamente a resolução espacial por meio de pooling e convoluções com stride. Seu objetivo é extrair features cada vez mais abstratas e semanticamente ricas, aumentando o campo receptivo efetivo e capturando contexto global da imagem. Também reduz o custo computacional.

Upsampling (decoder): recupera a resolução espacial original, ampliando progressivamente os mapas de features até atingir as dimensões da imagem de entrada. Seu objetivo é produzir previsões densas pixel a pixel, combinando as features de alto nível do encoder com a localização espacial necessária para segmentação precisa.

Juntos, formam a arquitetura encoder-decoder típica: o encoder comprime e aprende representações, o decoder expande e localiza essas representações de volta ao espaço da imagem.

Questão 10: Upsampling do volume $[[5,6],[3,2]]$ para uma saída 4×4 (e 3×3 no item d).

a) Nearest Neighbor (Vizinho mais Próximo):

Cada pixel do input é replicado para preencher um bloco 2×2 na saída:

5	5	6	6
5	5	6	6
3	3	2	2
3	3	2	2

Cada valor de entrada é copiado para os 4 pixels correspondentes no espaço ampliado.

b) Bed of Nails (Cama de Pregos):

Cada valor do input é inserido no canto superior esquerdo de seu bloco 2×2, e os demais pixels são preenchidos com zero:

5	0	6	0
0	0	0	0
3	0	2	0
0	0	0	0

c) Max Unpooling:

A matriz 4×4 original do max pooling correspondente é:

3	1	1	4
2	1	3	2
3	7	1	7
3	2	2	4

O max pooling 2×2 selecionou os máximos e suas posições:

- Quadrante superior esquerdo (linhas 0-1, colunas 0-1): max = 3, posição (0,0)
- Quadrante superior direito (linhas 0-1, colunas 2-3): max = 4, posição (0,3)
- Quadrante inferior esquerdo (linhas 2-3, colunas 0-1): max = 7, posição (2,1)
- Quadrante inferior direito (linhas 2-3, colunas 2-3): max = 7, posição (2,3)

No max unpooling, os valores do volume de entrada [5, 6, 3, 2] são inseridos nas posições originais dos máximos, e os demais são zerados:

5	0	0	6
0	0	0	0
0	3	0	2
0	0	0	0

(5 → pos (0,0); 6 → pos (0,3); 3 → pos (2,1); 2 → pos (2,3))

d) Convolução Transposta — saída 3×3 (padding=0, stride=1):

Filtro utilizado:

0	1
2	3

Para cada elemento da entrada, multiplica-se pelo filtro e acumula-se na posição correspondente da saída:

- Entrada (0,0) = 5: contribui com $5 \times \text{filtro}$ nas posições (0,0)...(1,1): [0, 5; 10, 15]
- Entrada (0,1) = 6: contribui com $6 \times \text{filtro}$ nas posições (0,1)...(1,2): [0, 6; 12, 18]
- Entrada (1,0) = 3: contribui com $3 \times \text{filtro}$ nas posições (1,0)...(2,1): [0, 3; 6, 9]
- Entrada (1,1) = 2: contribui com $2 \times \text{filtro}$ nas posições (1,1)...(2,2): [0, 2; 4, 6]

Somando todas as contribuições posição a posição:

0	5	6
10	30	20
6	13	6

Cálculo da posição central (1,1): 15 (de entrada 0,0) + 0 (de 0,1) + 12 (de 0,1) + 3 (de 1,0) + 0 (de 1,1) = 30 .